



Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- The Need for Databases
- Data Models
- Relational Databases
- Database Design
- Storage Manager
- Query Processing
- Transaction Manager

Module No.	Unit No.	Contents	Hrs	CO
1	Introduction to Databases:		06	CO1
	1.1	Introduction, Purpose of Database Systems, DBMS system architecture , Data Models, Data independence		
	1.2	Database languages, Database Users and Administrator, Different types of Database Systems		

DBMS

- are software systems used to store, retrieve, and run queries on data.
- A DBMS serves as an interface between an end-user and a database, allowing users to create, read, update, and delete data in the database.
- a computerized data-keeping system.
- Software tool used to perform various types of operations on Data in database

DBMS contains information about a particular enterprise

Collection of interrelated data

Set of programs to access the data

An environment that is both *convenient* and *efficient* to use



DBMS



- It provides protection and security to the database.
- In the case of multiple users, it also maintains data consistency.
- 1960 – Charles Bachman designed first DBMS system

Study Guide

Component of DBMS

- **User** : Application programmers, end user, database administrator
- **Software** : Controls the storage, management, retrieval of data in data base
- **Hardware** : Input device, output device, network device, storage device
- **Data** : data stored in database, numerical, non numerical or logical data





Advantages of DBMS

- **Controls database redundancy:** It can control data redundancy because it stores all the data in one single database file and that recorded data is placed in the database.
- **Data sharing:** In DBMS, the authorized users of an organization can share the data among multiple users.
- **Easily Maintenance:** It can be easily maintainable due to the centralized nature of the database system.
- **Reduce time:** It reduces development time and maintenance need.
- **Backup:** It provides backup and recovery subsystems which create automatic backup of data from hardware and software failures and restores the data if required.
- **multiple user interface:** It provides different types of user interfaces like graphical user interfaces, application program interfaces





Disadvantages of DBMS

- **Cost of Hardware and Software:** It requires a high speed of data processor and large memory size to run DBMS software.
- **Size:** It occupies a large space of disks and large memory to run them efficiently.
- **Higher impact of failure:** Failure is highly impacted the database because in most of the organization, all the data stored in a single database and if the database is damaged due to electric failure or database corruption then the data may be lost forever.



Database Management System (DBMS)

- **Database Applications:**
 - Banking: transactions
 - Airlines: reservations, schedules
 - Universities: registration, grades
 - Sales: customers, products, purchases
 - Online retailers: order tracking, customized recommendations
 - Manufacturing: production, inventory, orders, supply chain
 - Human resources: employee records, salaries, tax deductions
- Databases can be very large.
- Databases touch all aspects of our lives



University Database Example

- Application program examples
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts
- In the early days, database applications were built directly on top of file systems



Purpose of Database management systems

Database management systems were developed to handle the following difficulties of typical file-processing systems supported by conventional operating systems:

Drawbacks of using file systems to store data

- Data redundancy and inconsistency
 - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
 - Need to write a new program to carry out each new task
- Integrity problems
 - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones



Drawbacks of using file systems to store data (Cont.)

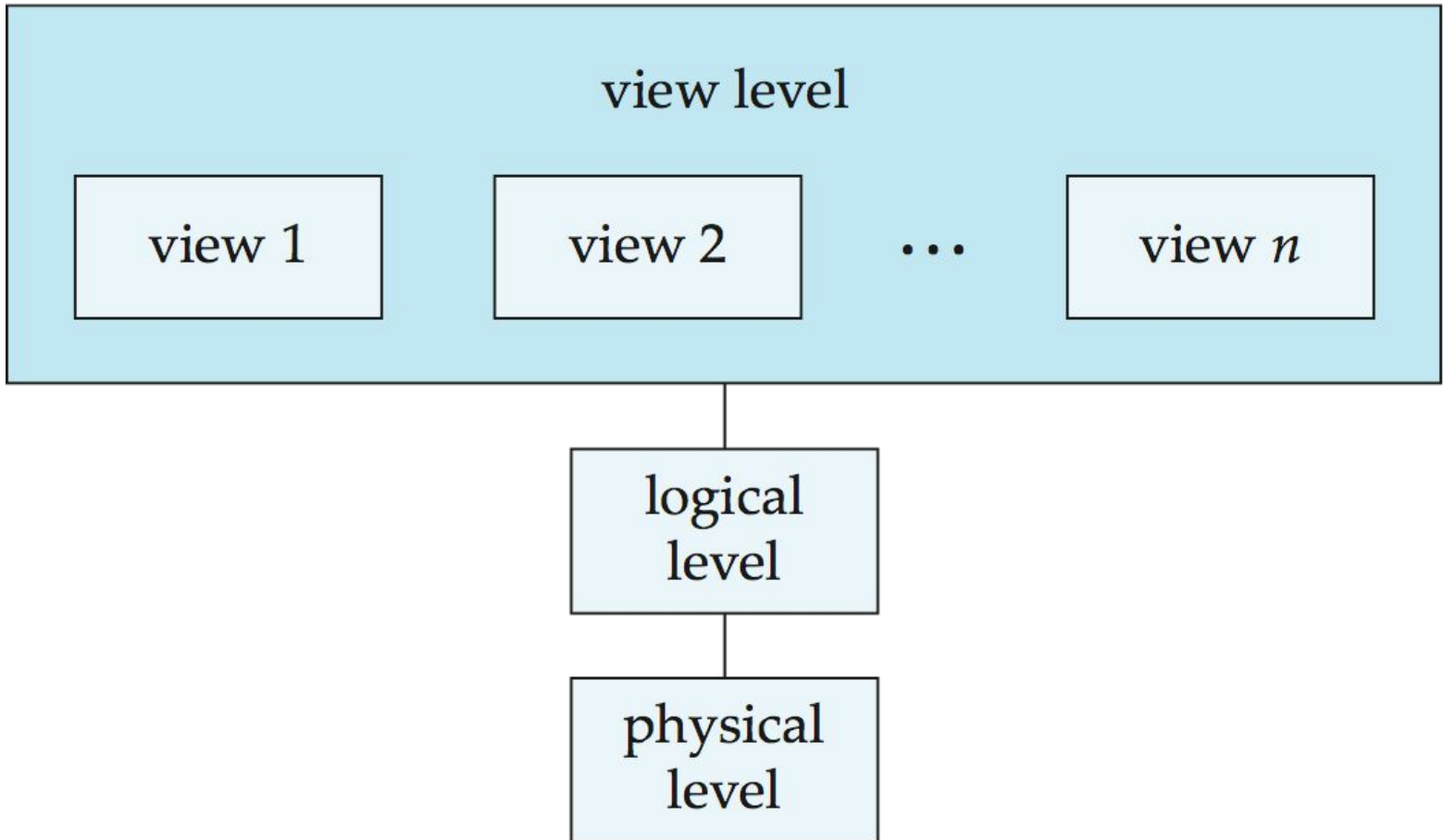
- Atomicity of updates
 - Failures may leave database in an inconsistent state with partial updates carried out
 - Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
 - Concurrent access needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - 4 Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
 - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems



View of Data

An architecture for a database system





Database Users

Users are differentiated by the way they expect to interact with the system.

Application programmers: interact with system through DML calls.

2. Application Programmers

- ☆ Computer **professionals**.
- ☆ Application development tools.
- ☆ Develop **user interfaces**.
- ☆ RAD.
- ☆ Web/Mobile interface.





Specialized users: write specialized database applications that do not fit into the traditional data processing framework ,

Among these applications are computer aided-design systems, knowledge-base and expert systems etc.

Sophisticated users: sophisticated users form their requests in a database query language and submit them to a query processor to make the storage manager understand the requests.

Analysts who submit queries to explore data in the database fall in this category.





1. Naive Users

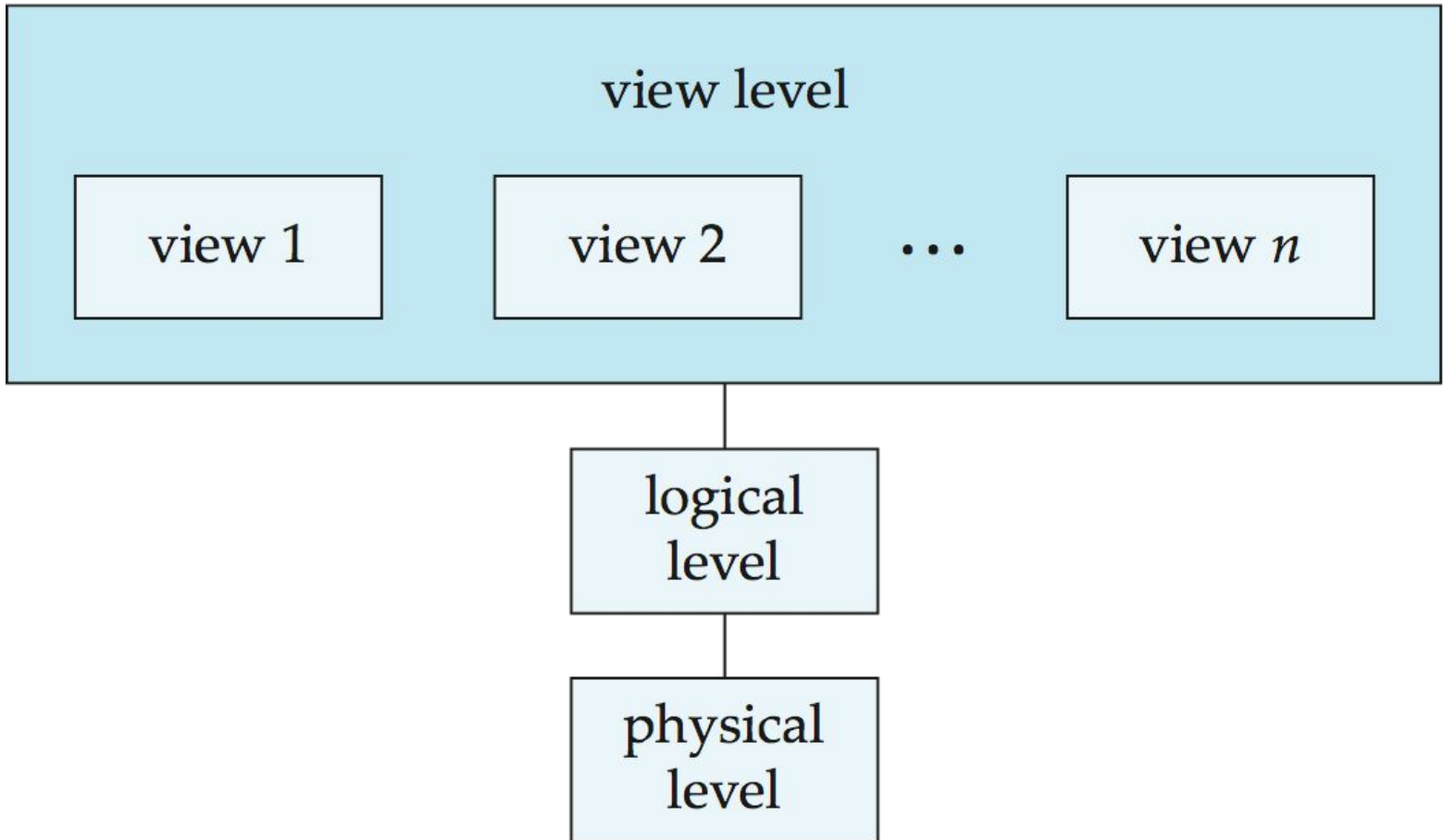
- ☆ Unsophisticated users.
- ☆ Invoke/use the application program.
- ☆ Ex: Clerk, Teller etc.
- ☆ Web/Mobile/UI interface.
- ☆ Desktop applications.





View of Data

An architecture for a database system





Levels of Abstraction

- **Physical level:** describes how a record (e.g. customer) is stored, uses complex data.
- **Logical level:** describes **what** data is stored in database, and the relationships among the data.

type customer = **record**

name: string;

street: string;

city: integer;

end;

- **View level:** application programs hide details of data types. Views can also hide information (e.g. salary) for security purposes.(GUI)



Instances and Schemas

- Similar to types and variables in programming languages
- Schema – the logical structure of the database (e.g., set of customers and accounts and the relationship between them)
- Instance – the actual content of the database at a particular point in time



Schemas

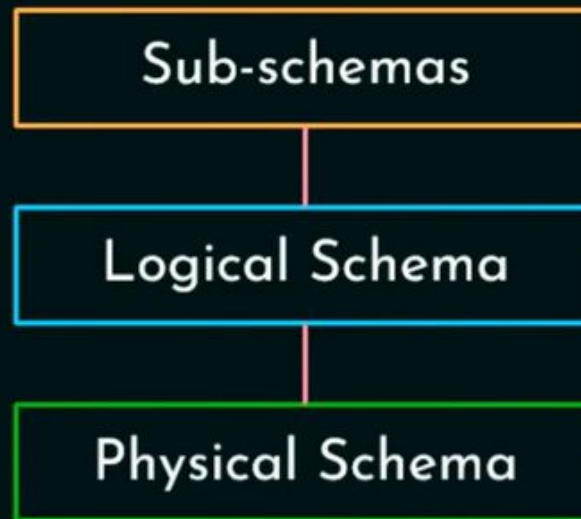
- ☆ The **overall design** of the database.
- ☆ Database schema = Variable declaration.
- ☆ **Type** definitions.
- ☆ **No frequent changes** or not at all.
- ☆ Example:

```
type Student = record
    Rollno : numeric (5);
    Name   : char (25);
    Class  : char (10);
end;
```



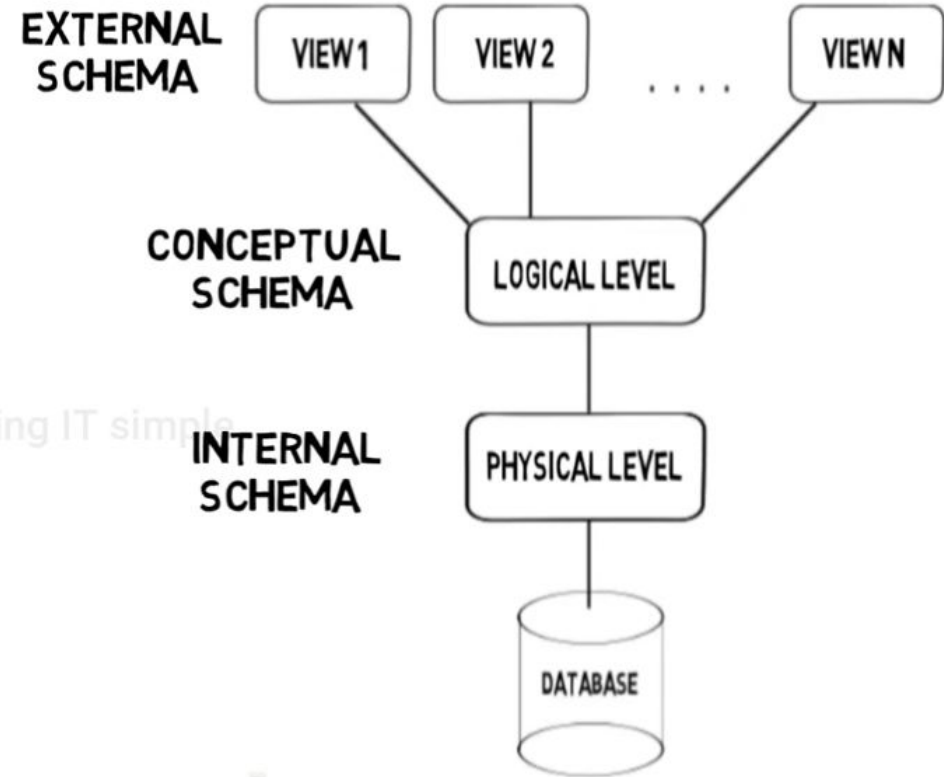
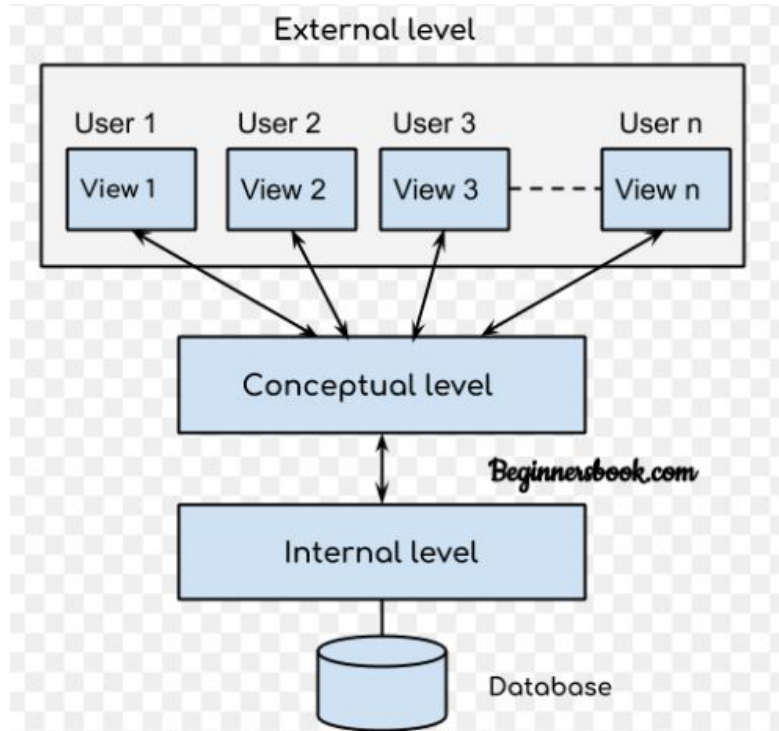
Instances

- ☆ The collection of information stored at a particular moment.
- ☆ Frequent changes.
- ☆ Database instant = Value of the variable.





3-Level Schema





If a database system is not multi-layered, then it becomes difficult to make any changes in the database system

Data Independence

- The capacity to change the schema at one level of a database system without the need to change the schema at the next higher level is referred to as Data Independence.
- The concept of Data Independence extends the concept of Three Schema Architecture.

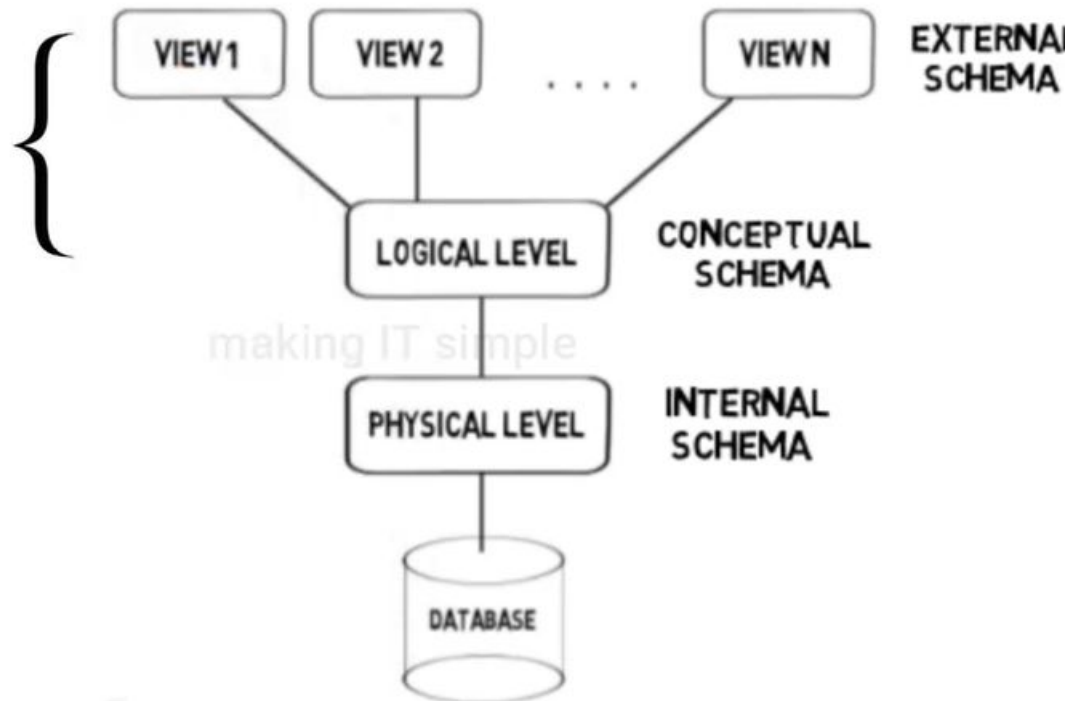


Data Independence

- Ability to modify a schema definition in one level without affecting a schema definition in the other levels.
- The interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.
- Two levels of data independence
 - Physical data independence
 - Logical data independence



**LOGICAL DATA
INDEPENDENCE**

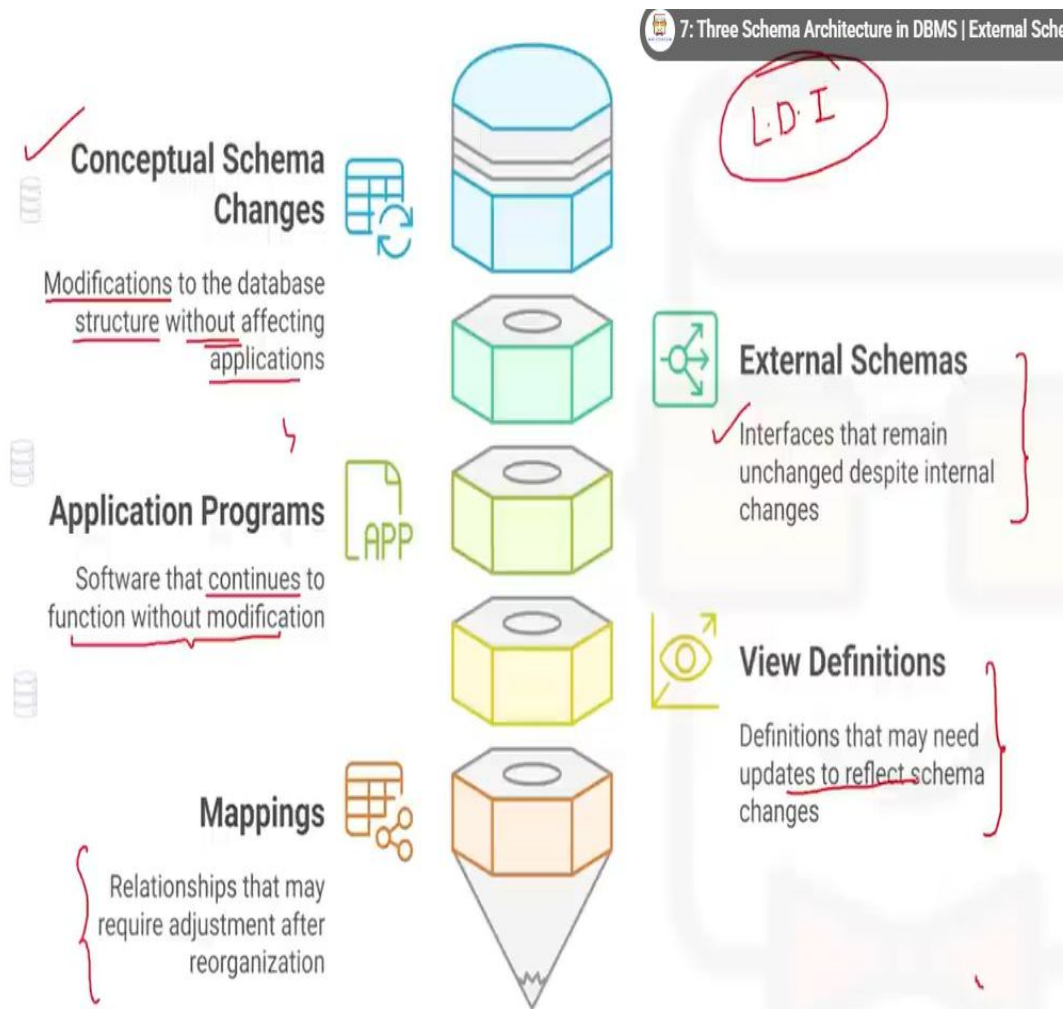




Data Independence conti..

- **Logical Data Independence**

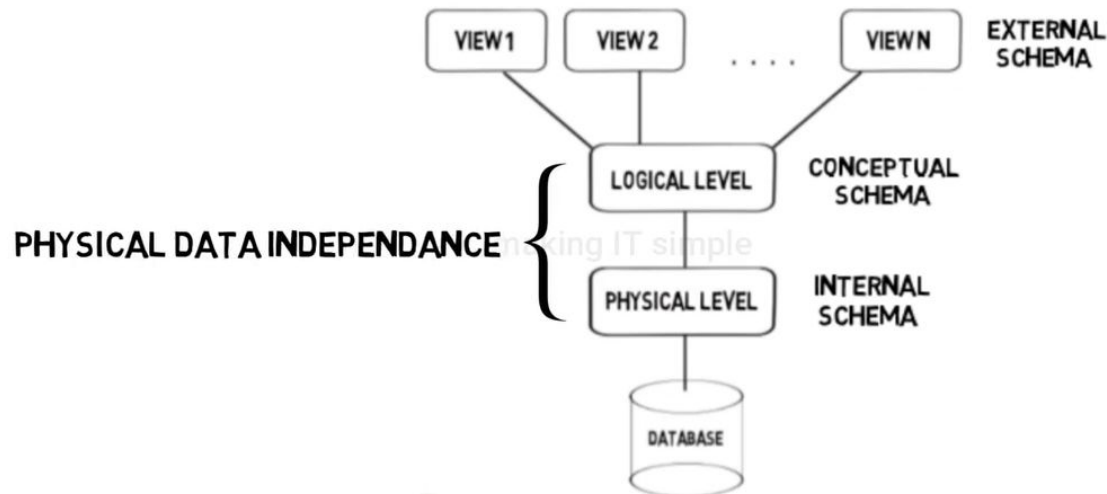
- Logical data is data about database, that is, it stores information about how data is managed inside. For example, a table (relation) stored in the database and all its constraints, applied on that relation.
- Logical data independence is a kind of mechanism, which liberalizes itself from actual data stored on the disk. If we do some changes on table format, it should not change the data residing on the disk.





● Physical Data Independence

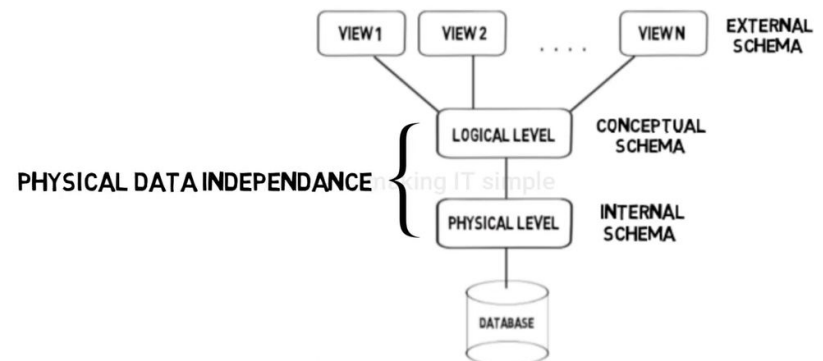
- All the schemas are logical, and the actual data is stored in bit format on the disk. Physical data independence is the power to change the physical data without impacting the schema or logical data.
- For example, in case we want to change or upgrade the storage system itself – suppose we want to replace hard-disks with SSD – it should not have any impact on the logical data or schemas.





Physical Data Independence

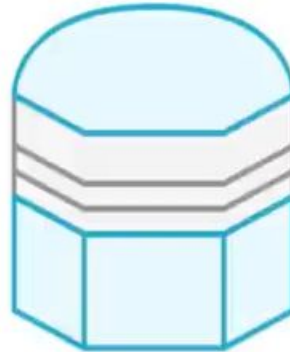
- Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. And, the external schemas need not be changed as well.
- Changes to the internal schema may be needed because some physical files were reorganized—for example, by creating additional access structures—to improve the performance of retrieval or update.
- If the same data as before remains in the database, we should not have to change the conceptual schema.





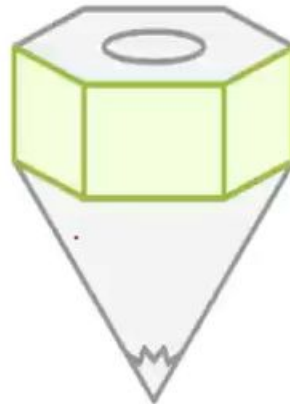
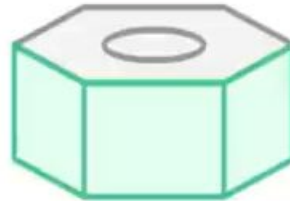
Internal Schema Changes

Modifications to physical files and structures



External Schema Impact

No changes needed for external schemas

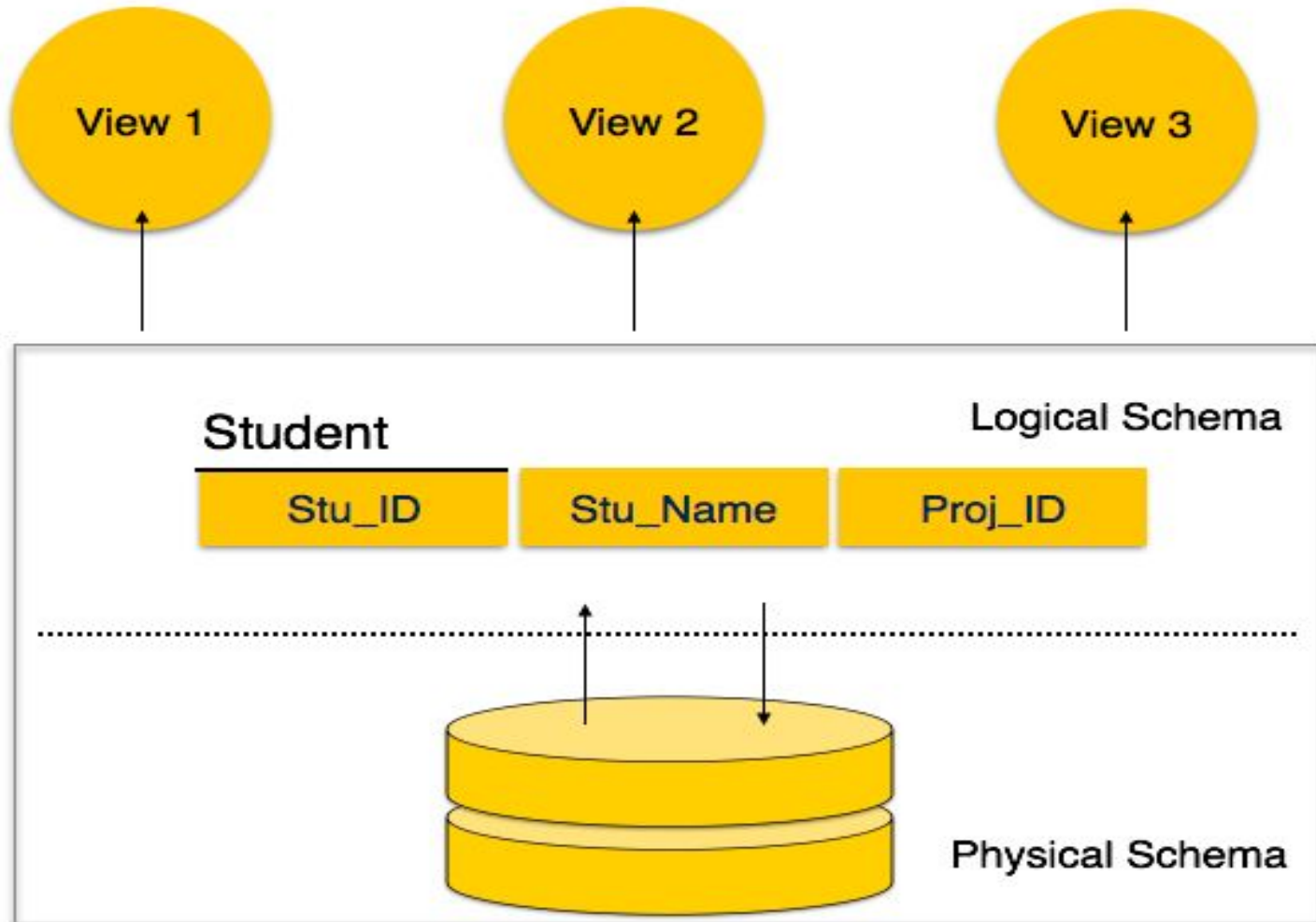


Conceptual Schema Stability

Maintaining the same data structure



Example





Database Languages

- DDL- Data Definition Language
- DDLs are used to define the metadata of the database. i.e.; using this, we create schema, tables, constraints, indexes in the database. DDLs are also used to modify Schema, tables index etc. Basically, using DDL statements, we create skeleton of the database. It helps to store the metadata information like number of schemas and tables, their names, columns in each table, indexes, constraints etc in the database.
- CREATE – to create database and its objects like (table, index, views, store procedure, function and triggers)
- ALTER – alters the structure of the existing database
- DROP – delete objects from the database
- TRUNCATE – remove all records from a table, including all spaces allocated for the records are removed
- COMMENT – add comments to the data dictionary
- RENAME – rename an object



Data Definition Language (DDL)

- Specification notation for defining the database schema
- DDL compiler generates a set of tables stored in a data dictionary
- Data dictionary contains ***metadata*** (data about data)
- Data storage and definition language – special type of DDL in which the storage structure and access methods used by the database system are specified



Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
- Two classes of languages
 - Procedural – user specifies what data is required and how to get those data
 - Nonprocedural – user specifies what data is required without specifying how to get those data



Data Manipulation Language (DML)

- DML- Data Manipulation Language
- When we have to insert records into table or get specific record from the table, or need to change some record, or delete some record or perform any other actions on records in the database, we need to have some media to perform it. DML helps to handle user requests. It helps to insert, delete, update, and retrieve the data from the database.
- SELECT – retrieve data from the a database
- INSERT – insert data into a table
- UPDATE – updates existing data within a table
- DELETE – Delete all records from a database table



DATABASE ADMINISTRATOR

- The life cycle of database starts from designing, implementing to administration of it.
- A database for any kind of requirement needs to be designed perfectly so that it should work without any issues.
- Once all the design is complete, it needs to be installed. Once this step is complete, users start using the database.
- The database grows as the data grows in the database.
- When the database becomes huge, its performance comes down. Also accessing the data from the database becomes challenge.
- There will be unused memory in database, making the memory inevitably huge. These administration and maintenance of database is taken care by database Administrator
- A DBA has many responsibilities. A good performing database is in the hands of DBA.



- **Installing and upgrading the DBMS Servers:** - DBA is responsible for installing a new DBMS server for the new projects. He is also responsible for upgrading these servers as there are new versions comes in the market or requirement. If there is any failure in upgradation of the existing servers, he should be able revert the new changes back to the older version, thus maintaining the DBMS working. He is also responsible for updating the service packs/ hot fixes/ patches to the DBMS servers.
- **Design and implementation:** - Designing the database and implementing is also DBA's responsibility. He should be able to decide proper memory management, file organizations, error handling, log maintenance etc for the database.
- **Performance tuning:** - Since database is huge and it will have lots of tables, data, constraints and indices, there will be variations in the performance from time to time. Also, because of some designing issues or data growth, the database will not work as expected. It is responsibility of the DBA to tune the database performance. He is responsible to make sure all the queries and programs works in fraction of seconds



- **Migrate database servers:** - Sometimes, users using oracle would like to shift to SQL server or Netezza. It is the responsibility of DBA to make sure that migration happens without any failure, and there is no data loss.
- **Backup and Recovery:** - Proper backup and recovery programs needs to be developed by DBA and has to be maintained him. This is one of the main responsibilities of DBA. Data/objects should be backed up regularly so that if there is any crash, it should be recovered without much effort and data loss.
- **Security:** - DBA is responsible for creating various database users and roles, and giving them different levels of access rights.
- **Documentation:** - DBA should be properly documenting all his activities so that if he quits or any new DBA comes in, he should be able to understand the database without any effort. He should basically maintain all his installation, backup, recovery, security methods. He should keep various reports about database performance.
- **There are different kinds of DBA depending on the responsibility that he owns**



Database Administrator

- Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs:
- Database administrator's duties include:
 - Schema definition
 - Storage structure and access method definition
 - Schema and physical organization modification
 - Granting user authority to access the database
 - Specifying integrity constraints
 - Acting as liaison with users
 - Monitoring performance and responding to changes in requirements



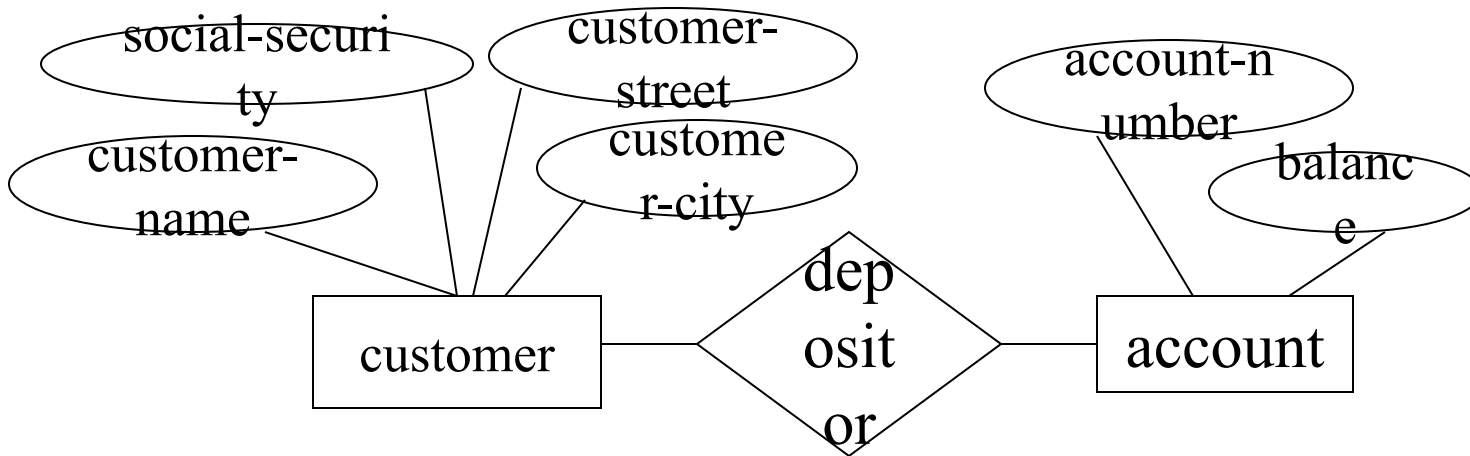
Data Models

- A collection of tools for describing:
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- Object-based logical models
 - Entity-relationship model
 - Object-oriented model
 - Semantic model
 - Functional model
- Record-based logical models
 - Relational model (e.g., SQL/DS, DB2)
 - Network model
 - Hierarchical model (e.g., IMS)



Entity-Relationship Model

Example of entity-relationship model





Relational Model

Example of tabular data in the relational model:

name	ssn	street	city	account-number
Johnson	192-83-7465	Alma	Palo Alto	A-101
Smith	019-28-3746	North	Rye	A-215
Johnson	192-83-7465	Alma	Palo Alto	A-201
Jones	321-12-3123	Main	Harrison	A-217
Smith	019-28-3746	North	Rye	A-201

account-number	balance
A-101	500
A-201	900
A-215	700
A-217	750



Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model

Columns

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows

(a) The *instructor* table



A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table











Levels of Abstraction

- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

type *instructor* = **record**

ID : string;

name : string;

dept_name : string;

salary : integer;

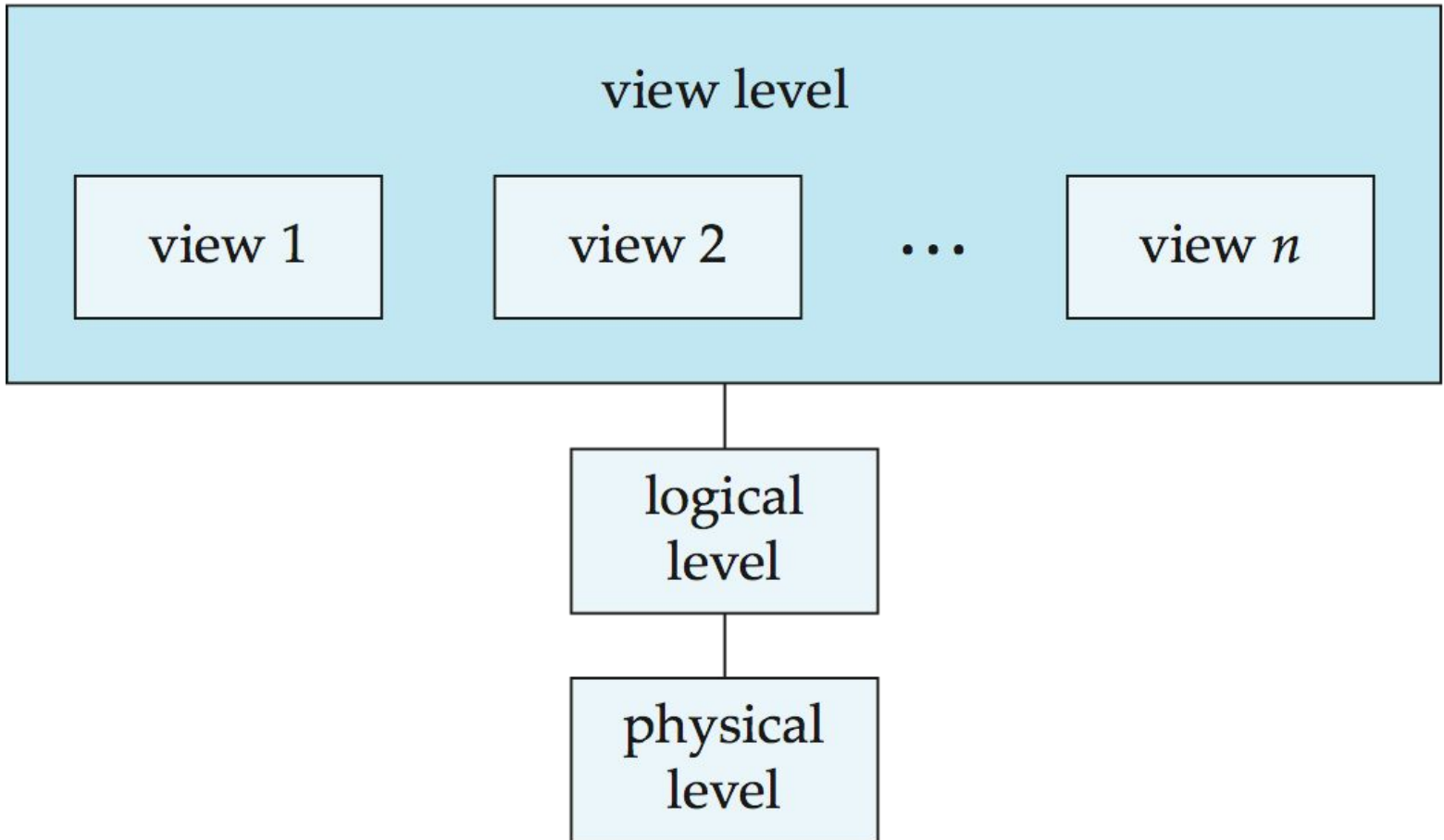
end;

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.



View of Data

An architecture for a database system





Instances and Schemas

- Similar to types and variables in programming languages
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - 4 Analogous to type information of a variable in a program
- **Physical schema**– the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable
- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema
 - Applications depend on the logical schema
 - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.



Data Models

- A collection of tools for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- Relational model
- Entity-Relationship data model (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semistructured data model (XML)
- Other older models:
 - Network model
 - Hierarchical model



Data Definition Language (DDL)

- Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a ***data dictionary***
- Data dictionary contains metadata (i.e., data about data)
 - Database schema
 - Integrity constraints
 - 4 Primary key (ID uniquely identifies instructors)
 - Authorization
 - 4 Who can access what



Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
 - DML also known as query language
- Two classes of languages
 - **Pure** – used for proving properties about computational power and for optimization
 - 4 Relational Algebra
 - 4 Tuple relational calculus
 - 4 Domain relational calculus
 - **Commercial** – used in commercial systems
 - 4 SQL is the most widely used commercial language



SQL

- The most widely used commercial language
- SQL is NOT a Turing machine equivalent language
- SQL is NOT a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database



Database Design

The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema.
Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database



Database Design (Cont.)

- Is there any problem with this relation?

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Approaches

- Need to come up with a methodology to ensure that each of the relations in the database is “good”
- Two ways of doing so:
 - Entity Relationship Model (Chapter 7)
 - 4 Models an enterprise as a collection of *entities* and *relationships*
 - 4 Represented diagrammatically by an *entity-relationship diagram*:
 - Normalization Theory (Chapter 8)
 - 4 Formalize what designs are bad, and test for them



Object-Relational Data Models

- Relational model: flat, “atomic” values
- Object Relational Data Models
 - Extend the relational data model by including object orientation and constructs to deal with added data types.
 - Allow attributes of tuples to have complex types, including non-atomic values such as nested relations.
 - Preserve relational foundations, in particular the declarative access to data, while extending modeling power.
 - Provide upward compatibility with existing relational languages.



XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange **data**, not just documents
- XML has become the basis for all new generation data interchange formats.
- A wide variety of tools is available for parsing, browsing and querying XML documents/data



Database Engine

- Storage manager
- Query processing
- Transaction manager



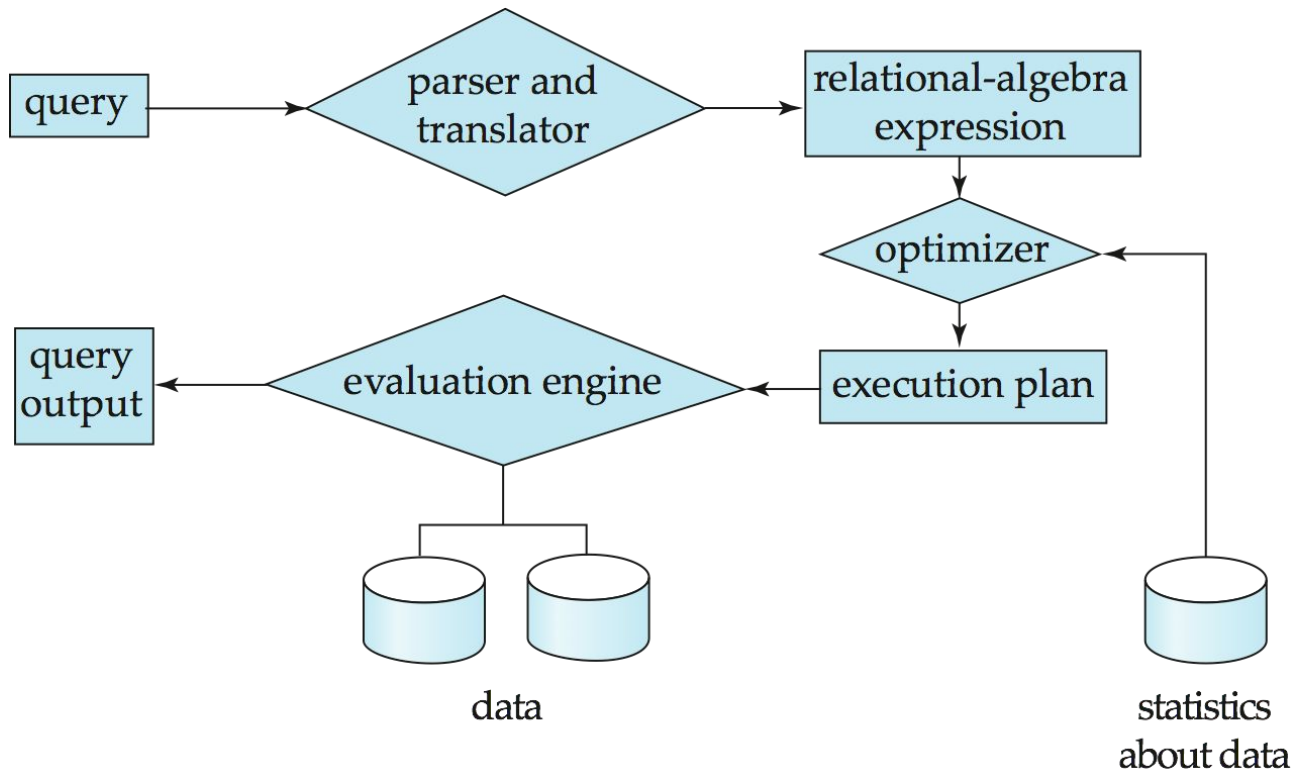
Storage Management

- **Storage manager** is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - Interaction with the OS file manager
 - Efficient storing, retrieving and updating of data
- Issues:
 - Storage access
 - File organization
 - Indexing and hashing



Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Query Processing (Cont.)

- Alternative ways of evaluating a given query
 - Equivalent expressions
 - Different algorithms for each operation
- Cost difference between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
 - Depends critically on statistical information about relations which the database must maintain
 - Need to estimate statistics for intermediate results to compute cost of complex expressions

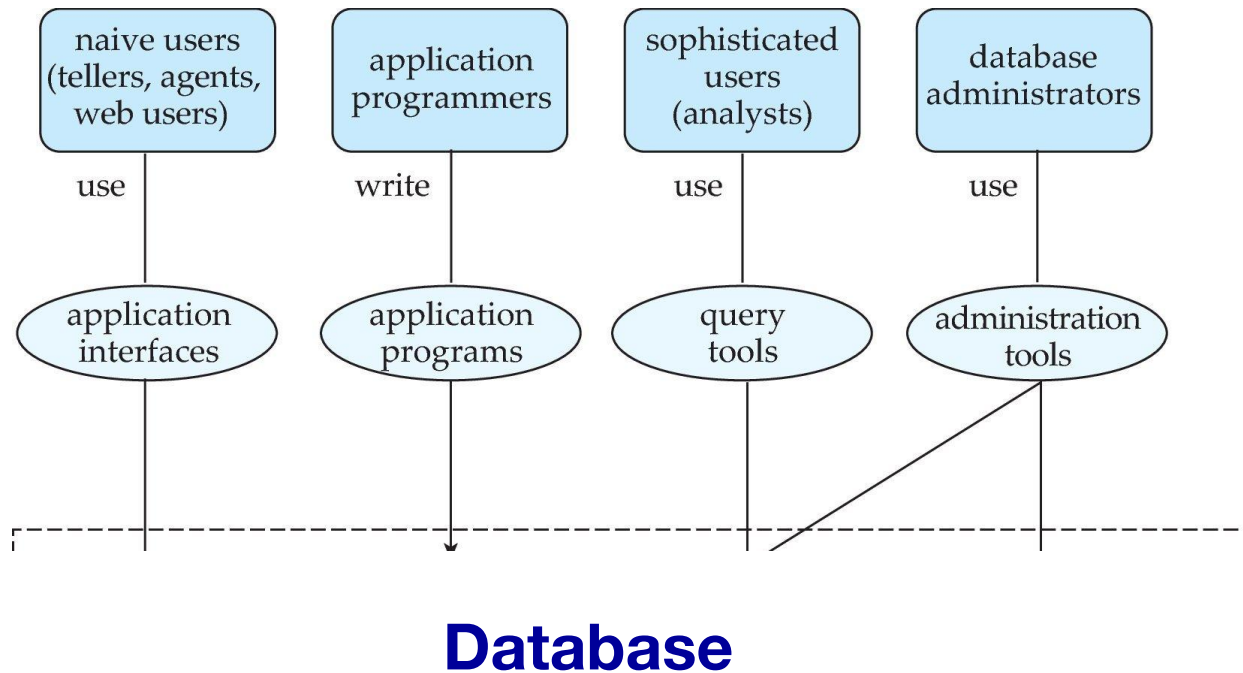


Transaction Management

- What if the system fails?
- What if more than one user is concurrently updating the same data?
- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

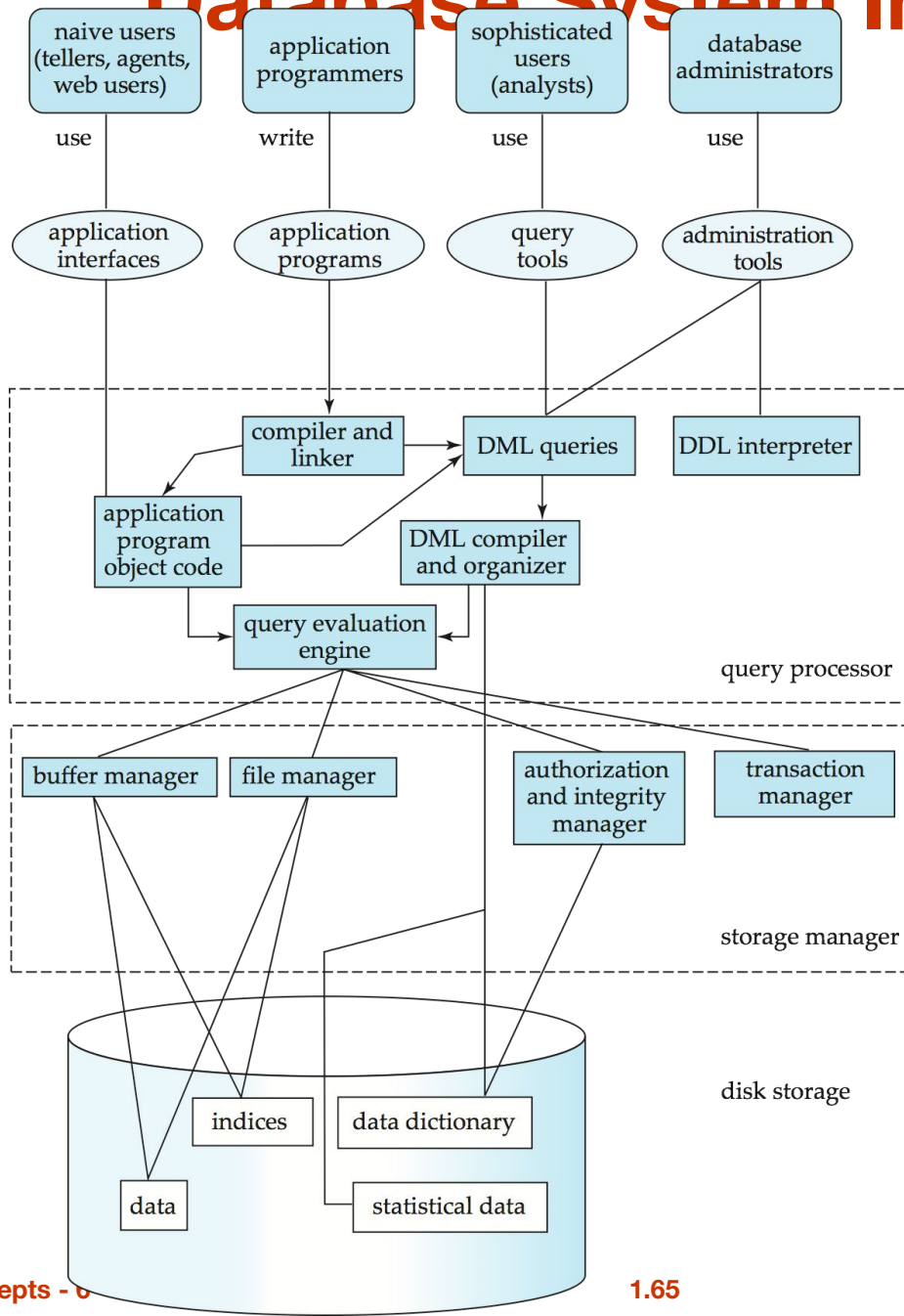


Database Users and Administrators





Database System Internals





Database Architecture

The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

- Centralized
- Client-server
- Parallel (multi-processor)
- Distributed



History of Database Systems

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - 4 Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - 4 Would win the ACM Turing Award for this work
 - 4 IBM Research begins System R prototype
 - 4 UC Berkeley begins Ingres prototype
 - High-performance (for the era) transaction processing



History (cont.)

- 1980s:
 - Research relational prototypes evolve into commercial systems
 - 4 SQL becomes industrial standard
 - Parallel and distributed database systems
 - Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce
- Early 2000s:
 - XML and XQuery standards
 - Automated database administration
- Later 2000s:
 - Giant data storage systems
 - 4 Google BigTable, Yahoo PNuts, Amazon, ..



End of Chapter 1



- Any attribute in the table which uniquely identifies each record in the table is called key. It can be a single attribute or a combination of attributes. For example, in STUDENT table, STUDENT_ID is a key, since it is unique for each student. In PERSON table, his passport number, driving license number, phone number, SSN, email address is keys since they are unique for each person.

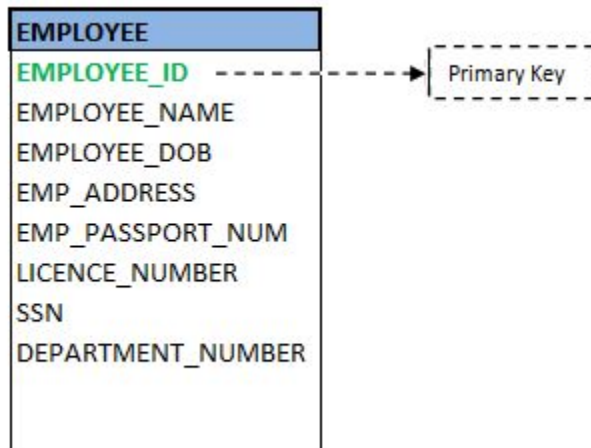
STUDENT
STUDENT_ID
STUDENT_NAME
ADDRESS
DOB
COURSE

PERSON
EMPLOYEE_NAME
EMPLOYEE_DOB
EMP_ADDRESS
EMP_PASSPORT_NUM
LICENCE_NUMBER
SSN



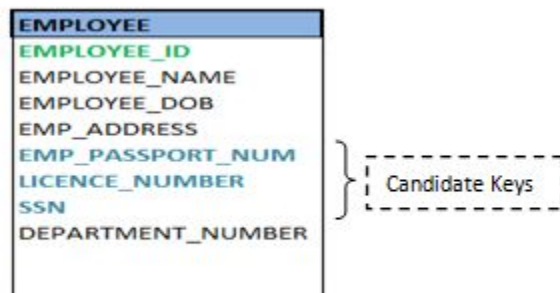
- Primary Key

It is the first and foremost key which is used to uniquely identify a record. It can be a single attribute or a combination of attributes. For an entity, there could be multiple keys as we saw in PERSON table. Most suitable key from those lists becomes a primary key. In the Person table above, we can select SSN as primary key, since it is unique for each person. We can even select Passport Number or license number as primary key as they are also unique for a person. However, selection of primary key for each entity is based on requirement and d



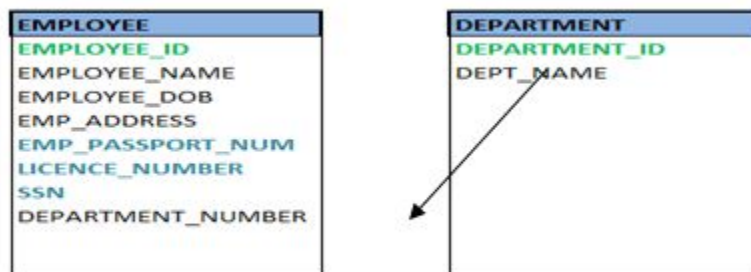


- Candidate Key
- As we discussed above, an employee is identified by his ID in his office. Apart from his ID, does he have any other unique keys, so that he can be identified from others? Yes, he has passport number, PAN number, SSN number (if applicable), driving license number, email address etc. These are also identifies specific person uniquely. But we can choose any one of these unique attribute as primary key in the table. Rest of the attributes, which holds as strong as primary key are considered as Candidate key/secondary key. In our example of employee table, EMPLOYEE_ID is best suited for primary key as its from his own employer. Rest of the attributes like passport number, SSN, license Number etc are considered as candidate key.





- Foreign key
- In a company there would be different departments - Accounting, Human Resource (HR), development, Quality, etc. An employee, who works for that company, works in specific department. But we know that employee and department are two different entities. So we cannot store his department information in employee table. Instead what we do is we link these two tables by means of primary key of one of the table i.e.; in this case, we pick the primary key of department table - DEPARTMENT_ID and add it as a new attribute/column in the Employee table. Now DEPARTMENT_ID is a foreign key for Employee table, and both the tables are related!





- Constraints are the conditions forced on the columns of the table to meet the data integrity. We have seen above what types of data integrities exists in the database. Now let see what constraints can be applied on tables so that data integrity is met.
- NOT NULL
- This constraint forces the column to have non-null value. We cannot enter/update any NULL value into such columns. It must have valid value all the time. For example, each student in STUDENT table should have class specified. No student can exist without class. Hence class column in the STUDENT table can be made NOT NULL.
- `CREATE TABLE STUDENT (STUDENT_ID NUMBER (10) NOT NULL
STUDENT_NAME VARCHAR2 (50) NOT NULL,
AGE NUMBER);`